

Timers



Timers

- O processo de contagem de tempo em microcontroladores pode ser implementado de modo bastante simples.
- Criar uma variável que será incrementada em intervalos regulares, seu valor indica quanto tempo passou.



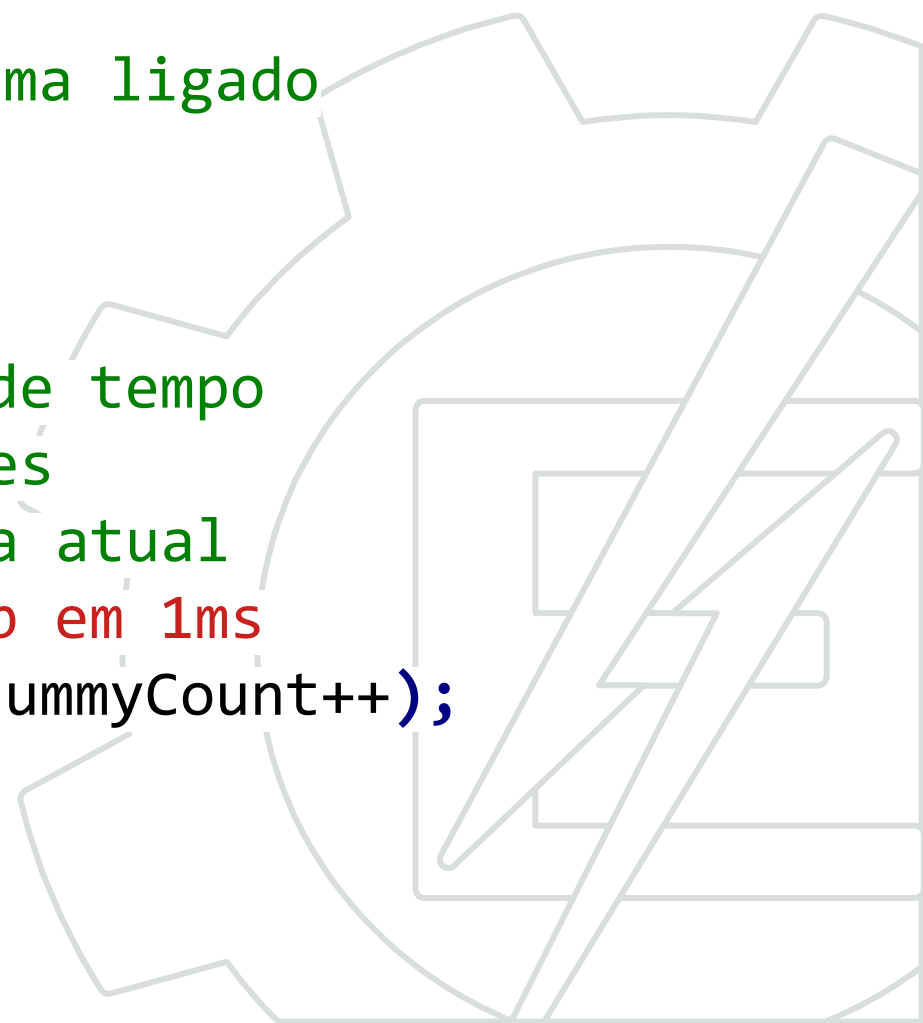
Exemplo de um retardo de tempo

```
...  
for(x=0;x<100;x++);  
...
```

Exemplo de uso:

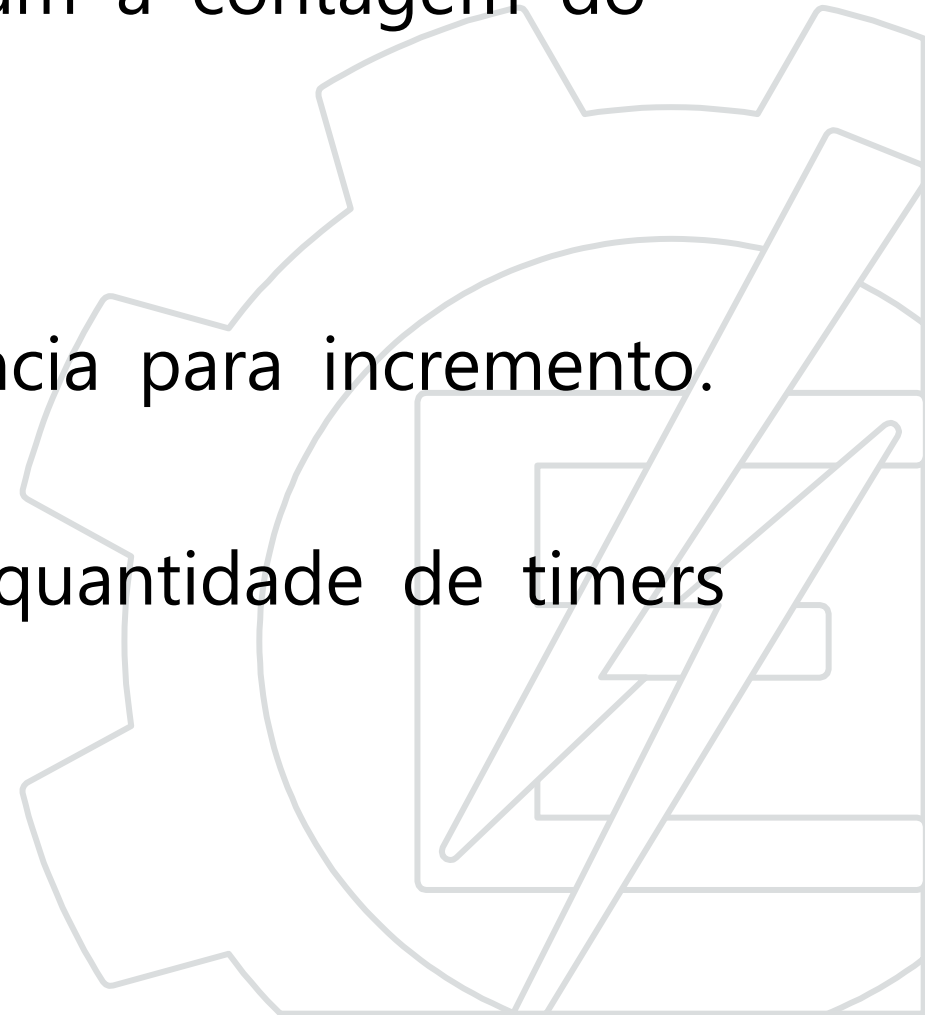
```
#include "lcd.h"
```

```
void main (void){  
    int time=0; //indica tempo com o sistema ligado  
    int dummyCount;  
    lcdInit();  
    for(;;){  
        time++; //incrementa a contagem de tempo  
        DoStuff(); //executa as atividades  
        timePrint(time); //imprime a hora atual  
        //tempo extra para acertar o loop em 1ms  
        for(dummyCount=0;dummyCount<134;dummyCount++);  
    }  
}
```

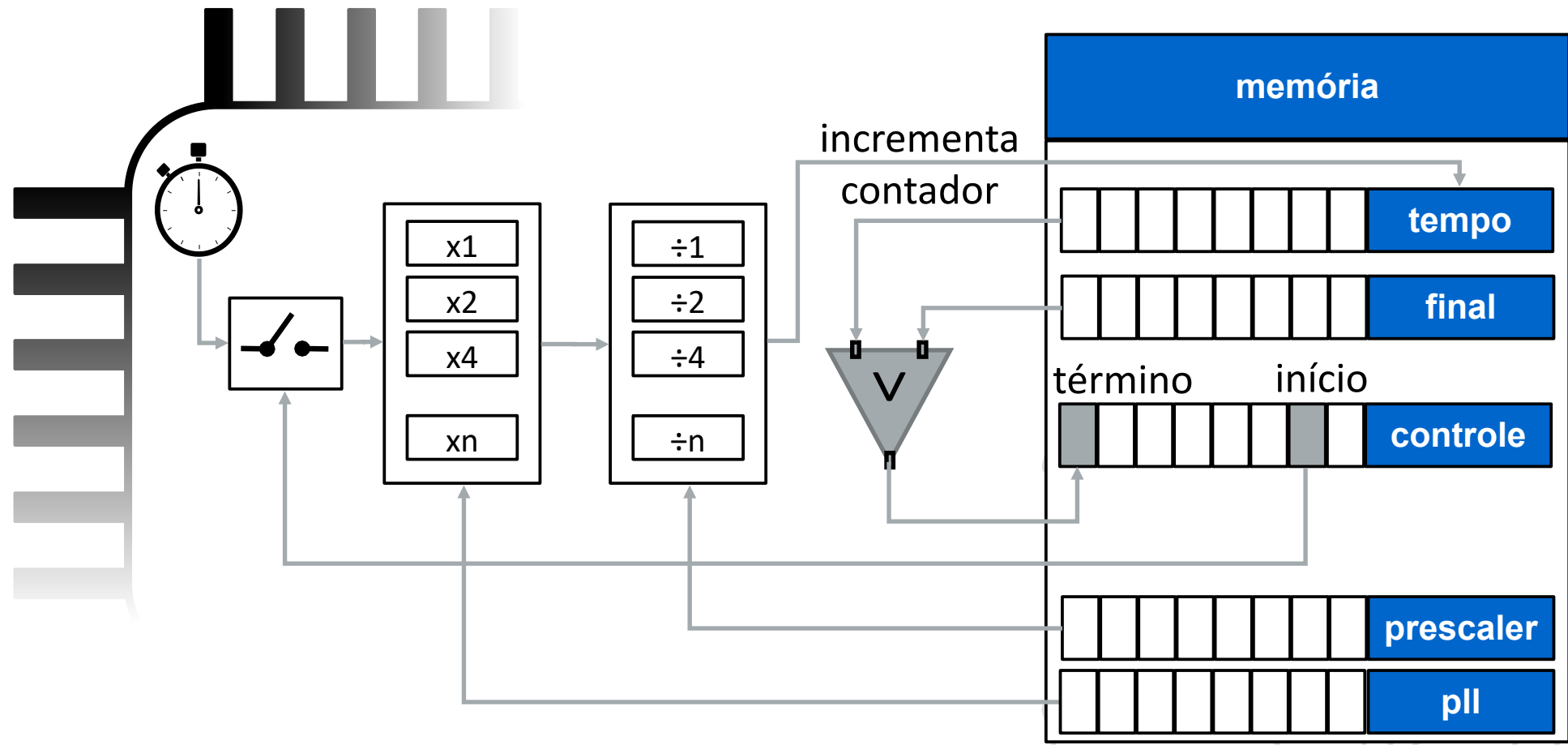


Timer por hardware

- Timers são circuitos dedicados que realizam a contagem do tempo.
- Geralmente utilizam o sinal do clock ou múltiplos/submúltiplos deste como referência para incremento.
- Todo microcontrolador possui uma certa quantidade de timers disponíveis.

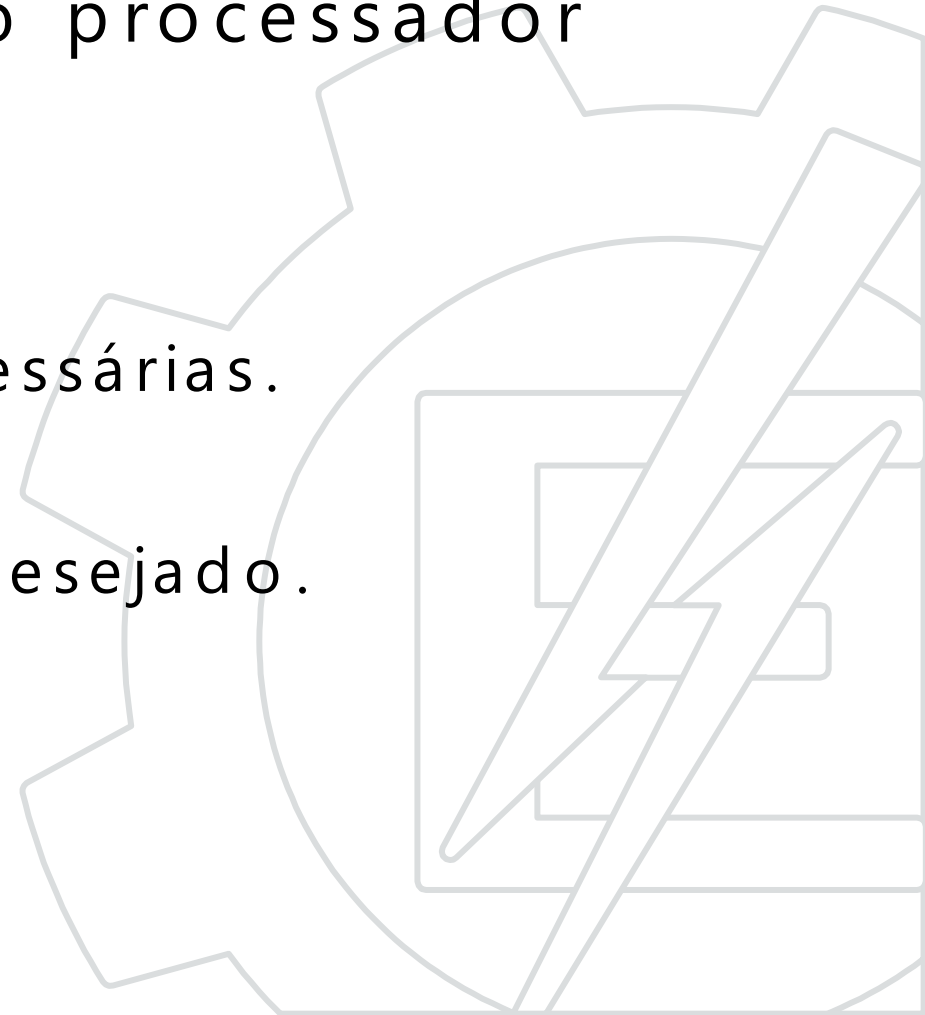


Timer por hardware



Timer por hardware

- A temporização por hardware não paralisa o programa como o delay, assim o processador continua fazendo outras tarefas.
- Modo de operação
 - Inicializam-se as configurações necessárias.
 - Prescaler.
 - Fonte de clock.
 - Configura-se o timer com o valor desejado.
 - Dispara o hardware.
 - Aguarda overflow.
 - Reinicia o timer.



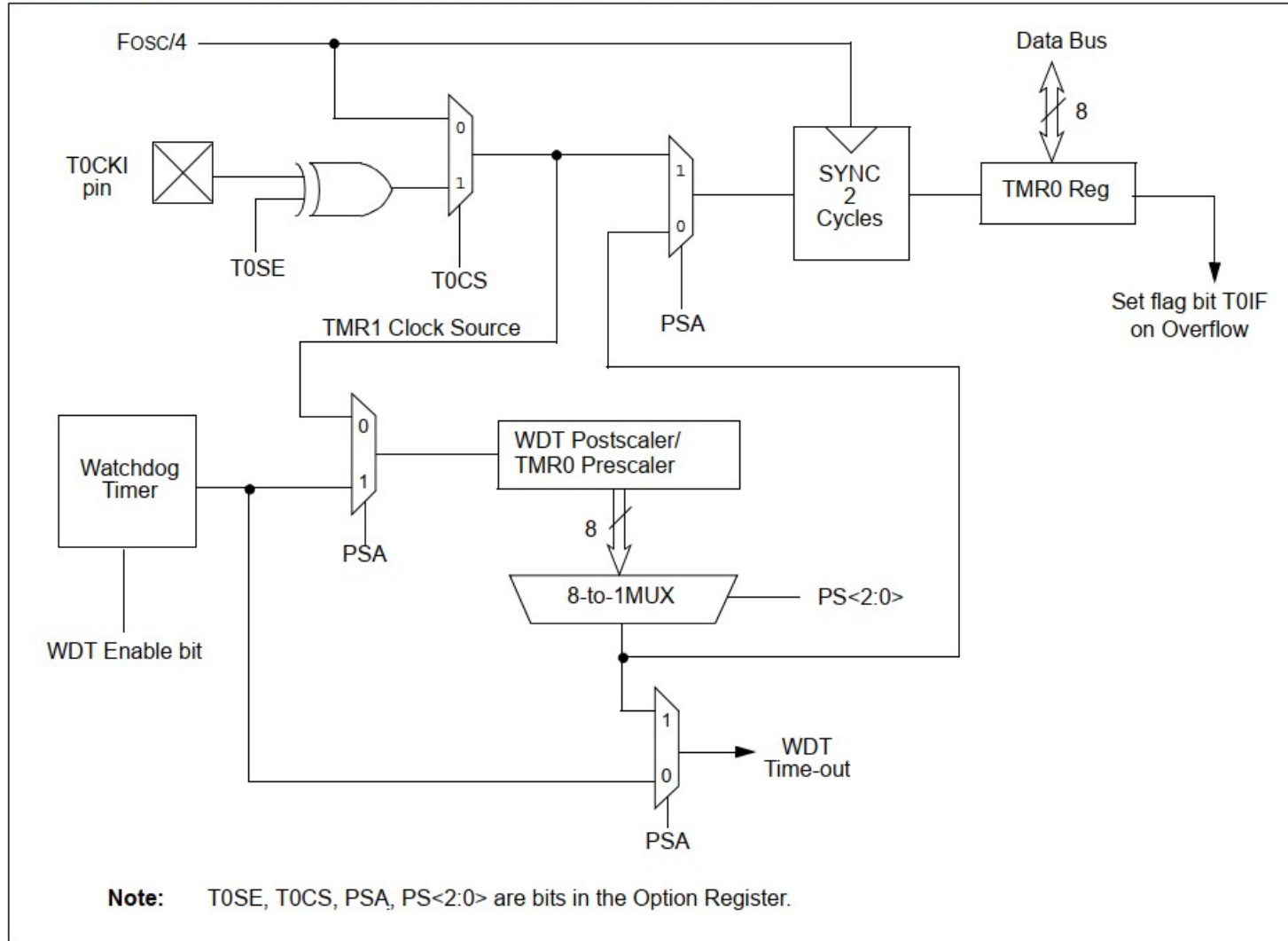
Timer no PIC16F628A

- O PIC16F628A possui 3 circuitos de timer internos
 - Timer0 – 8 bits
 - Timer1 – 16 bits
 - Timer2 – 16 bits



Timer no PIC16F628A

FIGURE 6-1: BLOCK DIAGRAM OF THE TIMER0/WDT



Registradores

- INTCON
- TMR0
- OPTION

A configuração é feita por ajustes em registradores (memória).

Timer no PIC16F628A

Indirect addr. (1)	00h
TMR0	01h
PCL	02h
STATUS	03h
FSR	04h
PORTA	05h
PORTB	06h
	07h
	08h
	09h
PCLATH	0Ah
INTCON	0Bh
PIR1	0Ch
	0Dh
TMR1L	0Eh
TMR1H	0Fh
T1CON	10h

Indirect addr. (1)	80h
OPTION	81h
PCL	82h
STATUS	83h
FSR	84h
TRISA	85h
TRISB	86h
	87h
	88h
	89h
PCLATH	8Ah
INTCON	8Bh
PIE1	8Ch
	8Dh
PCON	8Eh
	8Fh
	90h

Indirect addr. (1)	100h
TMR0	101h
PCL	102h
STATUS	103h
FSR	104h
	105h
PORTB	106h
	107h
	108h
	109h
PCLATH	10Ah
INTCON	10Bh
	10Ch
	10Dh
	10Eh
	10Fh

Indirect addr. (1)	180h
OPTION	181h
PCL	182h
STATUS	183h
FSR	184h
	185h
TRISB	186h
	187h
	188h
	189h
PCLATH	18Ah
INTCON	18Bh
	18Ch
	18Dh
	18Eh
	18Fh

Timer no PIC16F628A

Vai para 1 quando ocorrer um estouro do TIMER

TABLE 6-1: REGISTERS ASSOCIATED WITH TIMER0

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on POR	Value on All Other Resets
01h, 101h	TMR0	Timer0 Module Register								xxxx xxxx	uuuu uuuu
0Bh, 8Bh, 10Bh, 18Bh	INTCON	GIE	PEIE	T0IE	INTE	RBIE	T0IF	INTF	RBIF	0000 000x	0000 000u
81h, 181h	OPTION ⁽²⁾	$\overline{\text{RBPU}}$	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0	1111 1111	1111 1111
85h	TRISA	TRISA7	TRISA6	TRISA5	TRISA4	TRISA3	TRISA2	TRISA1	TRISA0	1111 1111	1111 1111

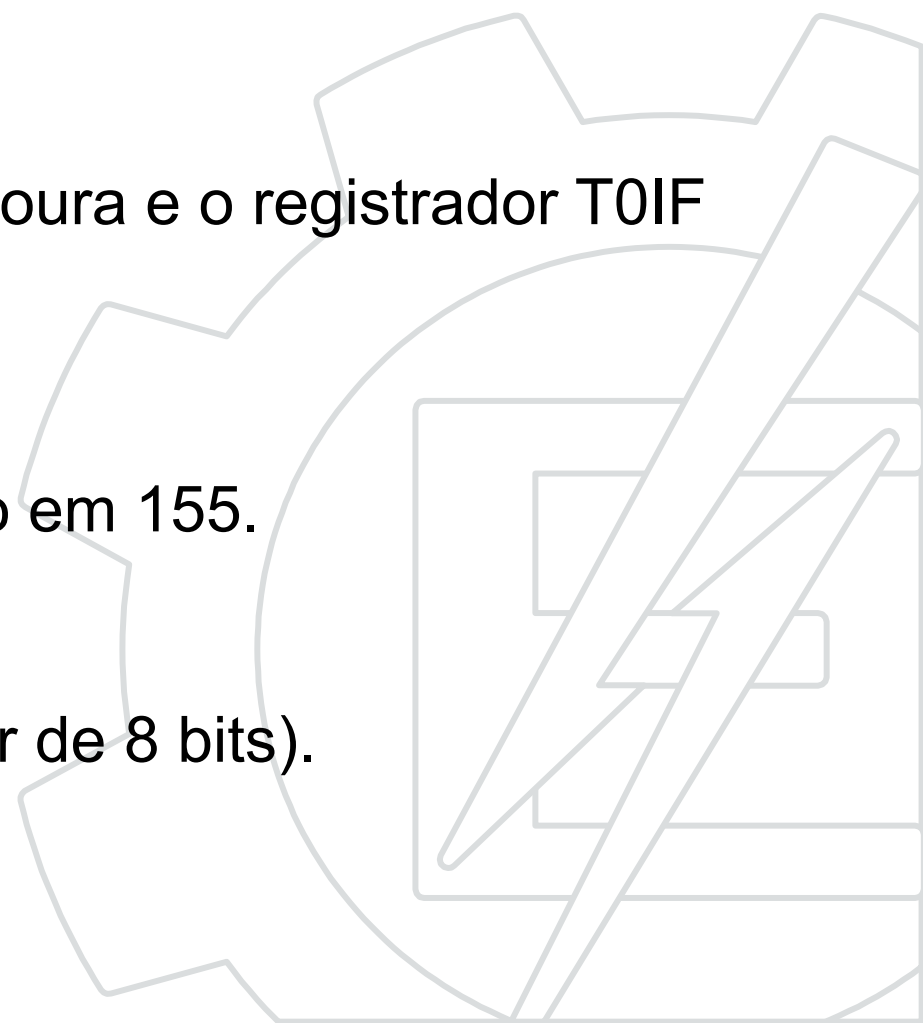
Legend: - = Unimplemented locations, read as '0', u = unchanged, x = unknown. Shaded cells are not used for Timer0.

Note 1: Option is referred by OPTION_REG in MPLAB[®] IDE Software.

Um bit sinaliza o fim da contagem

Timer no PIC16F628A

Exemplo: Vamos supor que o contador do Timer0 foi inicializado em 155

- Quando atingir 255 (máximo valor em 8 bits) ele estoura e o registrador T0IF irá para 1.
 - Neste exemplo ele conta de 0 a 100 pois foi iniciado em 155.
 - O valor da contagem sempre termina em 255 (se for de 8 bits).
- 

Timer no PIC16F628A

TMR0 armazena o valor da contagem do Timer

TABLE 6-1: REGISTERS ASSOCIATED WITH TIMER0

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on POR	Value on All Other Resets
01h, 101h	TMR0	Timer0 Module Register								xx x xxxx	uuuu uuuu
0Bh, 8Bh, 10Bh, 18Bh	INTCON	GIE	PEIE	T0IE	INTE	RBIE	T0IF	INTF	RBIF	0000 000x	0000 000u
81h, 181h	OPTION ⁽²⁾	$\overline{\text{RBPU}}$	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0	1111 1111	1111 1111
85h	TRISA	TRISA7	TRISA6	TRISA5	TRISA4	TRISA3	TRISA2	TRISA1	TRISA0	1111 1111	1111 1111

Legend: - = Unimplemented locations, read as '0', u = unchanged, x = unknown. Shaded cells are not used for Timer0.

Note 1: Option is referred by OPTION_REG in MPLAB[®] IDE Software.

Para contar até 10 devemos ajustar TMR0 com 245

Timer no PIC16F628A

TABLE 6-1: REGISTERS ASSOCIATED WITH TIMER0

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Value on POR	Value on All Other Resets
01h, 101h	TMR0	Timer0 Module Register								xxxx xxxx	uuuu uuuu
0Bh, 8Bh, 10Bh, 18Bh	INTCON	GIE	PEIE	T0IE	INTE	RBIE	T0IF	INTF	RBIF	0000 000x	0000 000u
81h, 181h	OPTION ⁽²⁾	$\overline{\text{RBPU}}$	INTEDG	T0CS	T0SE	PSA	PS2	PS1	PS0	1111 1111	1111 1111
85h	TRISA	TRISA7	TRISA6	TRISA5	TRISA4	TRISA3	TRISA2	TRISA1	TRISA0	1111 1111	1111 1111

Legend: - = Unimplemented locations, read as '0', u = unchanged, x = unknown. Shaded cells are not used for Timer0.

Note 1: Option is referred by OPTION_REG in MPLAB® IDE Software.

PS2,PS1,PS0 = Ajusta o prescaler (um divisor que aumenta o tempo).

PSA = Define se o circuito prescaler está ligado no circuito WatchDog ou no TIMER0.

T0SE = Define borda de subida ou descida.

T0CS = Define se o clock será interno ou virá do pino externo (RB4).

Timer no PIC16F628A

PS0,PS1 e PS2 permitem as seguintes configurações:

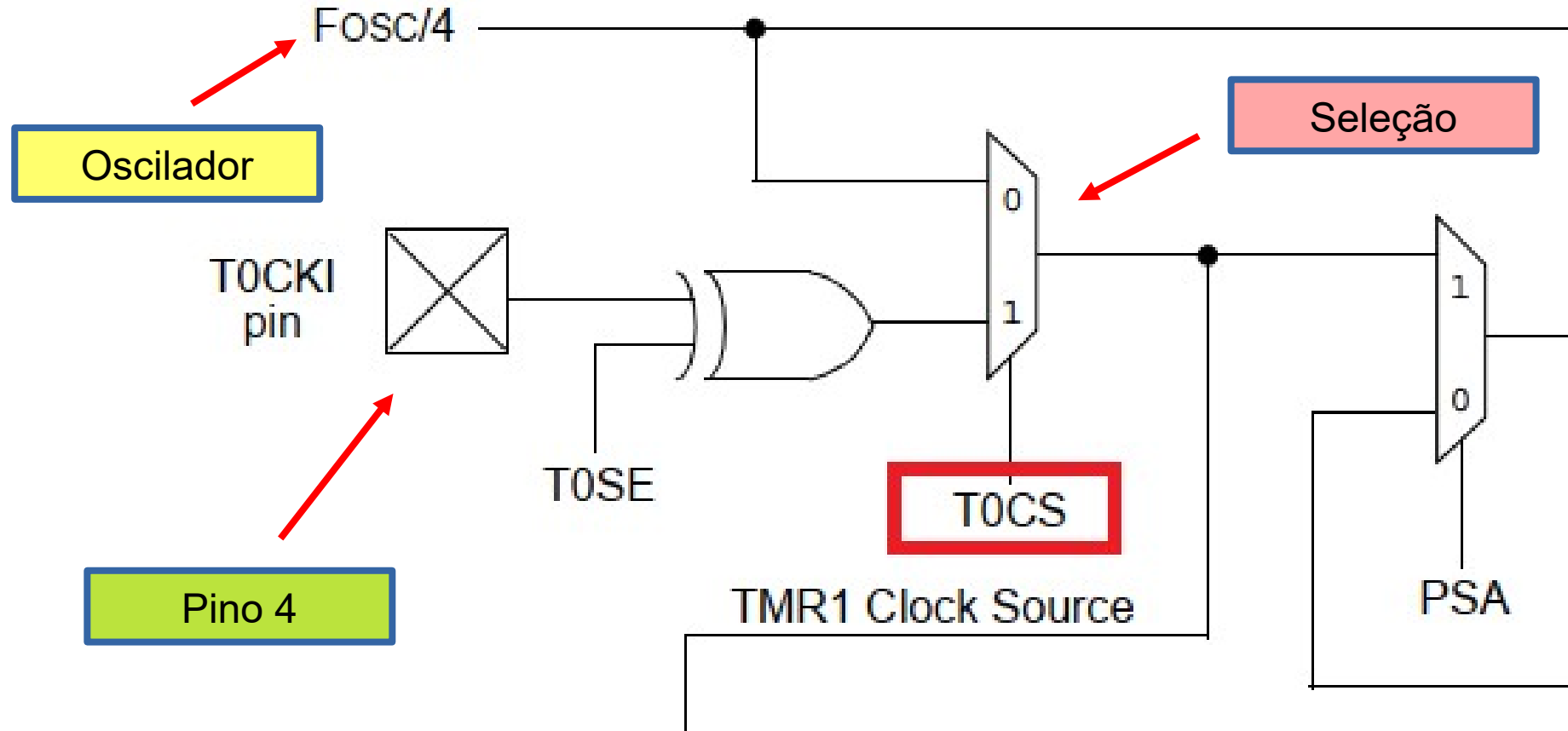
- 000 = 2 no prescaler
- 001 = 4 no prescaler
- 010 = 8 no prescaler
- 011 = 16 no prescaler
- 100 = 32 no prescaler
- 101 = 64 no prescaler
- 110 = 128 no prescaler
- 111 = 255 no prescaler

O Prescaler é um divisor de frequência

O clock do processador é muito rápido para maioria das aplicações

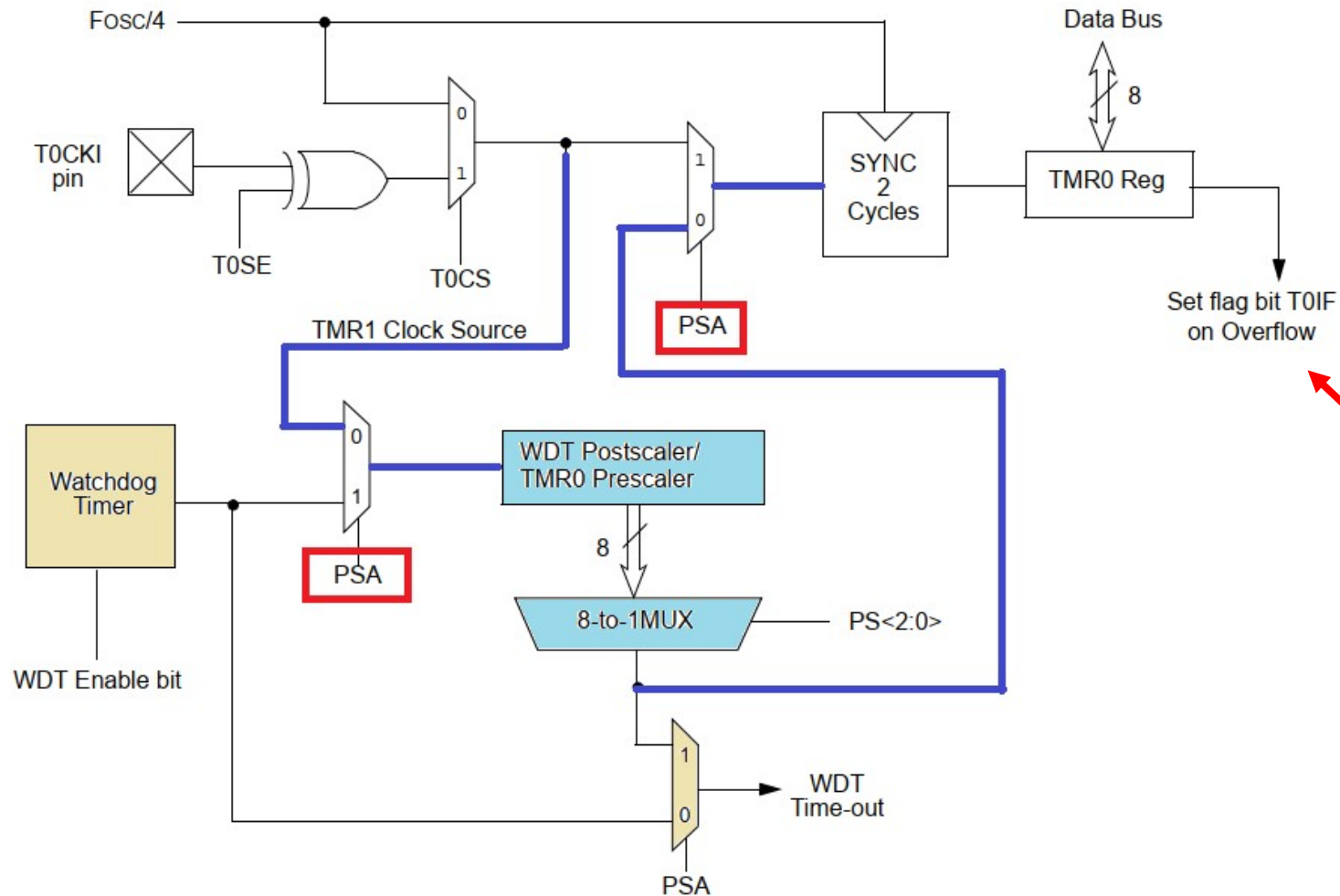
Timer no PIC16F628A

T0CS define se o clock vem do pino 4 ou do oscilador do PIC



Timer no PIC16F628A

PSA define se o Prescaler é ligado no WatchDog ou no TIMER0



TMR0 indica quando a contagem chegou ao fim

Timer o PIC16F628A

Calculando o tempo:

O tempo de cada clock é:

$$T_{clock} = \frac{1}{F_{osc}/4}$$

Considerando um ajuste inicial qualquer em TMR0 temos:

$$T_{contagem} = \frac{1}{F_{osc}/4} * (255 - TMR0)$$



Timer no PIC16F628A

Supondo um oscilador de 4MHz e TMR0 ajustado para 0

$$T_{contagem} = \frac{1}{F_{osc}/4} * (255 - TMR0)$$

Resultando em **255µs**.

Esse é o maior tempo possível sem prescaler.



Timer no PIC16F628A

Usando prescaler temos:

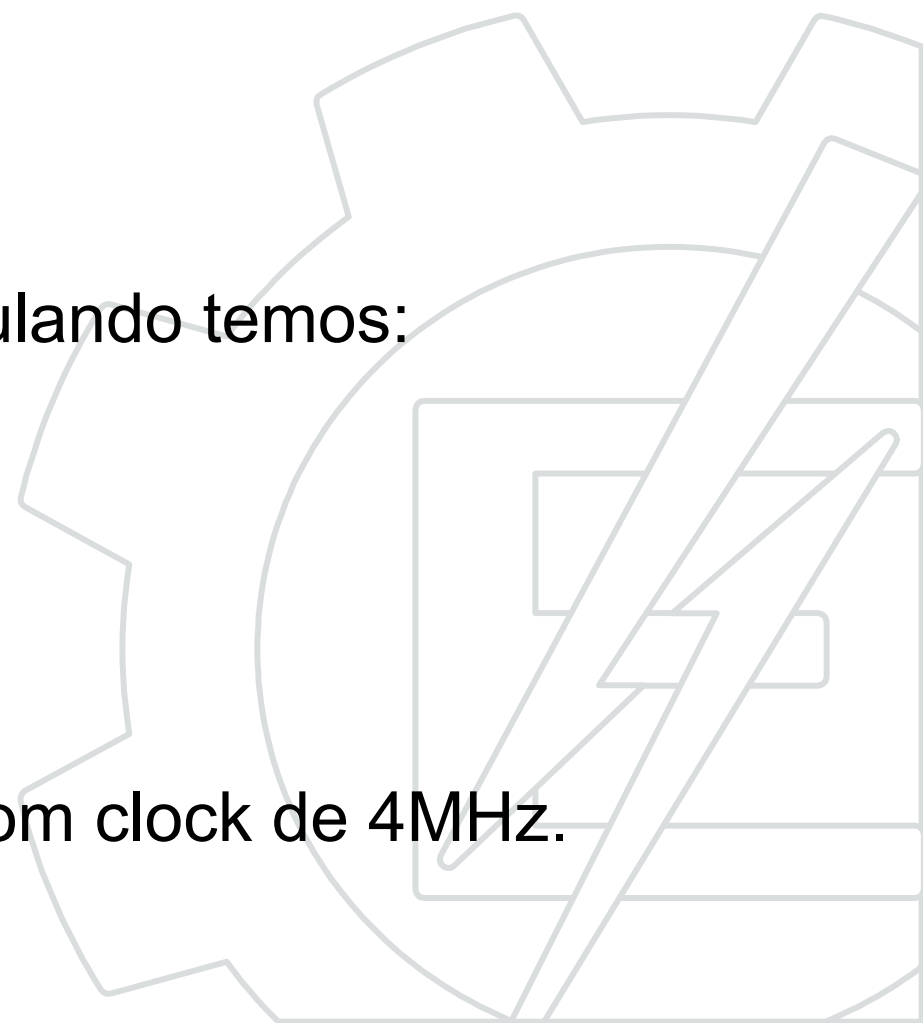
$$T_{contagem} = \frac{1}{F_{osc}/4} * (255 - TMR0) * Prescaler$$

O maior valor de Prescaler possível é 255, calculando temos:

$$T_{contagem} = \frac{1}{1.000.000} * (255 - 0) * 255$$

Resultando em **65ms**.

Esse é o maior tempo possível com prescaler com clock de 4MHz.



Timer no PIC16F628A

Usando um contador por software podemos contar o número de estouros.
Exemplo:

```
...
int cont=31; //ajustando para 31 estouros
ajusta_timer();
for(;;){
    if(T0IF){ //estourou
        T0IF = 0; //reseta o contador
        cont--; //decrementa o contador de estouros
        if(cont==0){
            ...
            cont=31;
        }
    }
    ... //executa outros comandos
}
```

Tempo aproximado = $31 \times 65\text{ms}$
2 segundos

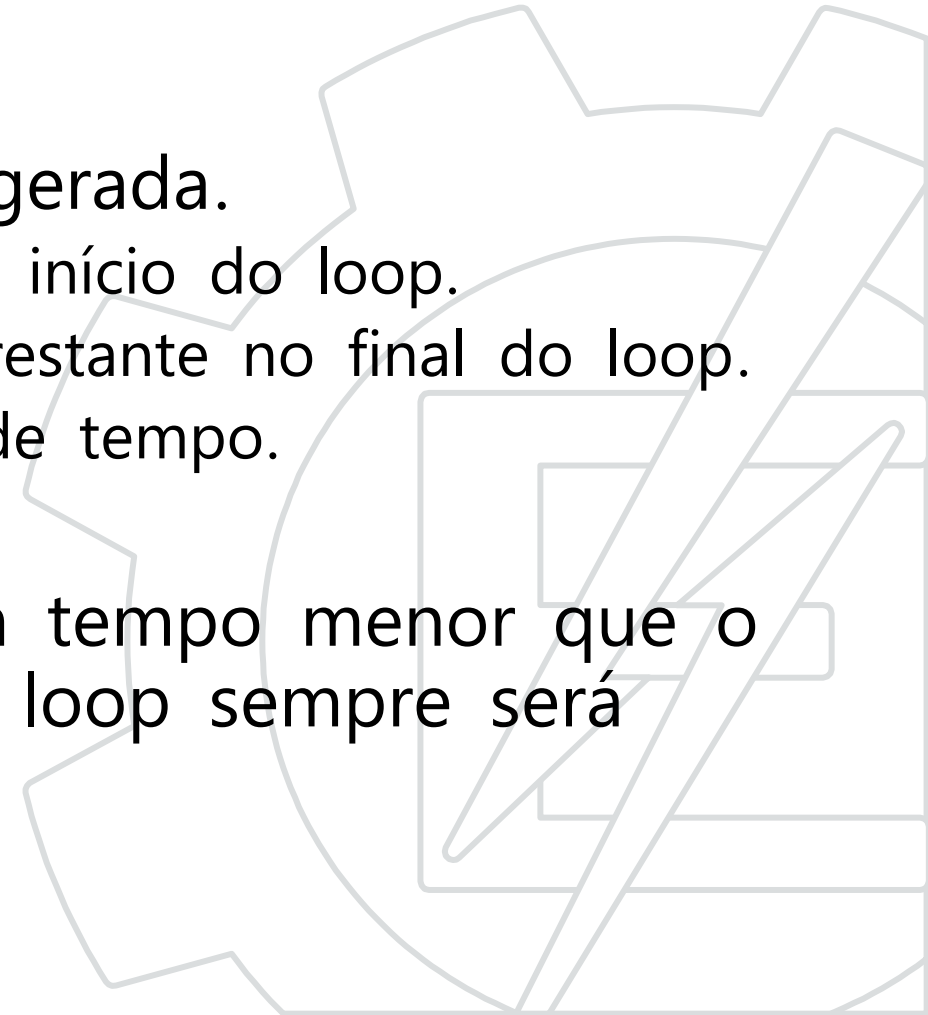
Criando uma base de tempo



Criando uma base de tempo

Utilizando o timer podemos resolver o problema de garantir a **temporização do loop principal**.

- Uma base de tempo confiável pode ser gerada.
 - Iniciando uma contagem com `timerStart()` no início do loop.
 - Usando `timerWait()` para aguardar o tempo restante no final do loop.
 - Deste modo o loop terá um limite mínimo de tempo.
- Se as atividades do loop consumirem um tempo menor que o mínimo, os tempos serão garantidos e o loop sempre será executado no tempo definido.



Exemplo:

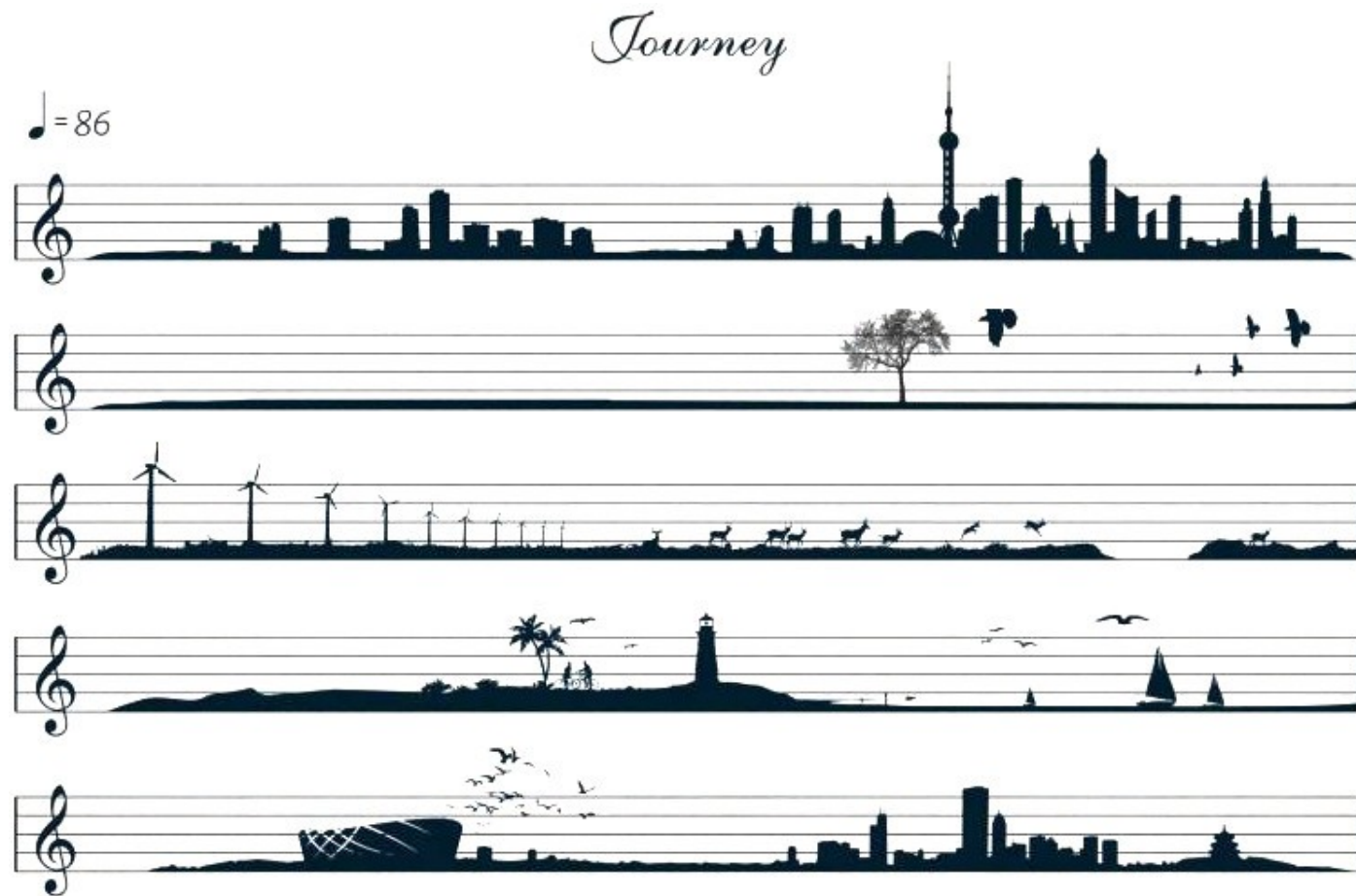
```
#include "lcd.h"
#include "timer.h"
void main (void){
    //armazena quanto tempo o sistema está ligado
    long int timeVar=0;
    int dummyCount;
    lcdInit();
    timerInit();
    for(;;){
        //configura o tempo do timer em 1ms
        timerStart(1000);
        timeVar++;
        DoStuff(); //executa as atividades
        //imprime a quantidade de segundos
        lcdNumber(timeVar/1000);
        //aguarda o tempo restante para 1ms
        timerWait();
    }
}
```



Reprodução de sons



Reprodução de sons



BOSE
Better sound through research®

Reprodução de sons

- A saída P W M permite várias frequências de trabalho.
- A frequência audível é de 20 à 20.000 Hz.
- Com esses valores optamos por trabalhar com as notas musicais da quinta e sexta oitavas de um piano, começando em C5 ou Dó Tenor que equivale à 523 Hz.
- Para representar a ausência de som será utilizado o valor de 125.000 Hz.
- A duração da nota será feita em múltiplos de 10ms, utilizando a biblioteca de timer como base de tempo.

```
#include "pwm.h"  
#include "timer.h"
```

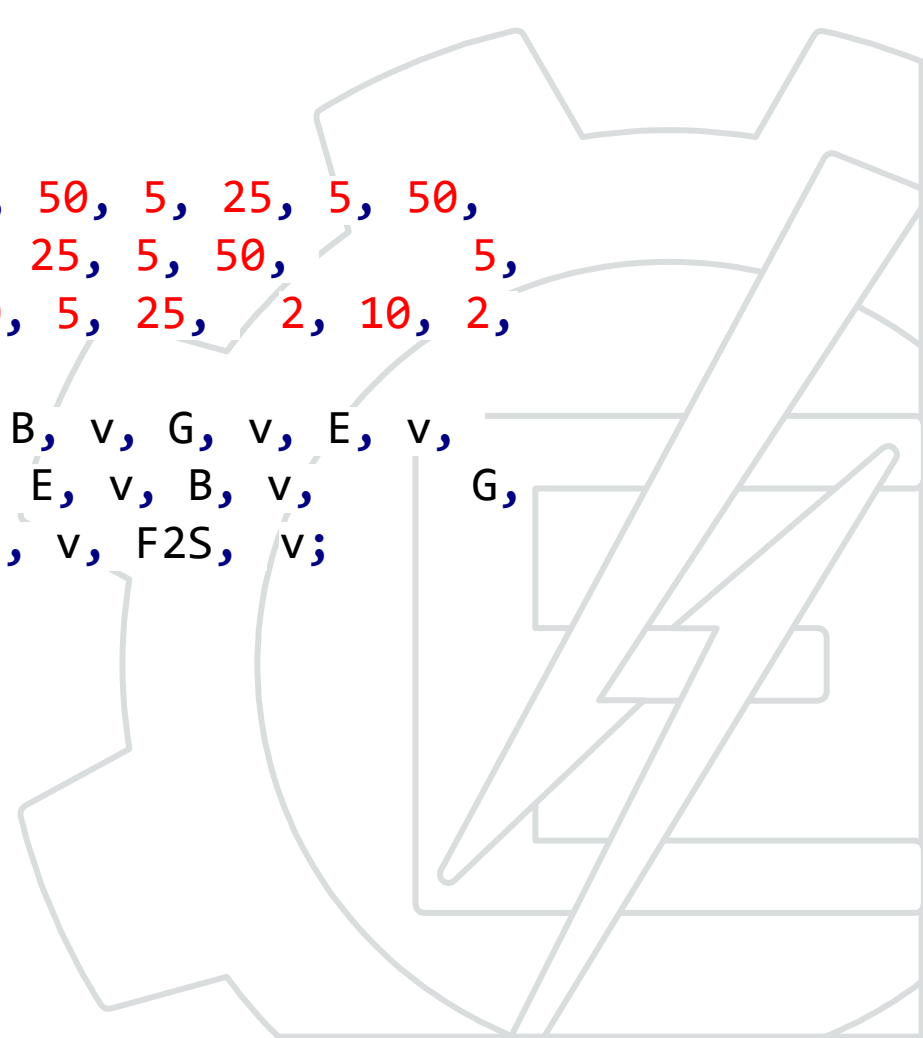
```
//frequência das notas musicais
```

```
#define C 523  
#define CS 554  
#define D 587  
#define DS 622  
#define E 659  
#define F 698  
#define FS 740  
#define G 784  
#define GS 830  
#define A 880  
#define AS 932  
#define B 987  
#define C2 2*C  
#define C2S 2*CS  
...
```

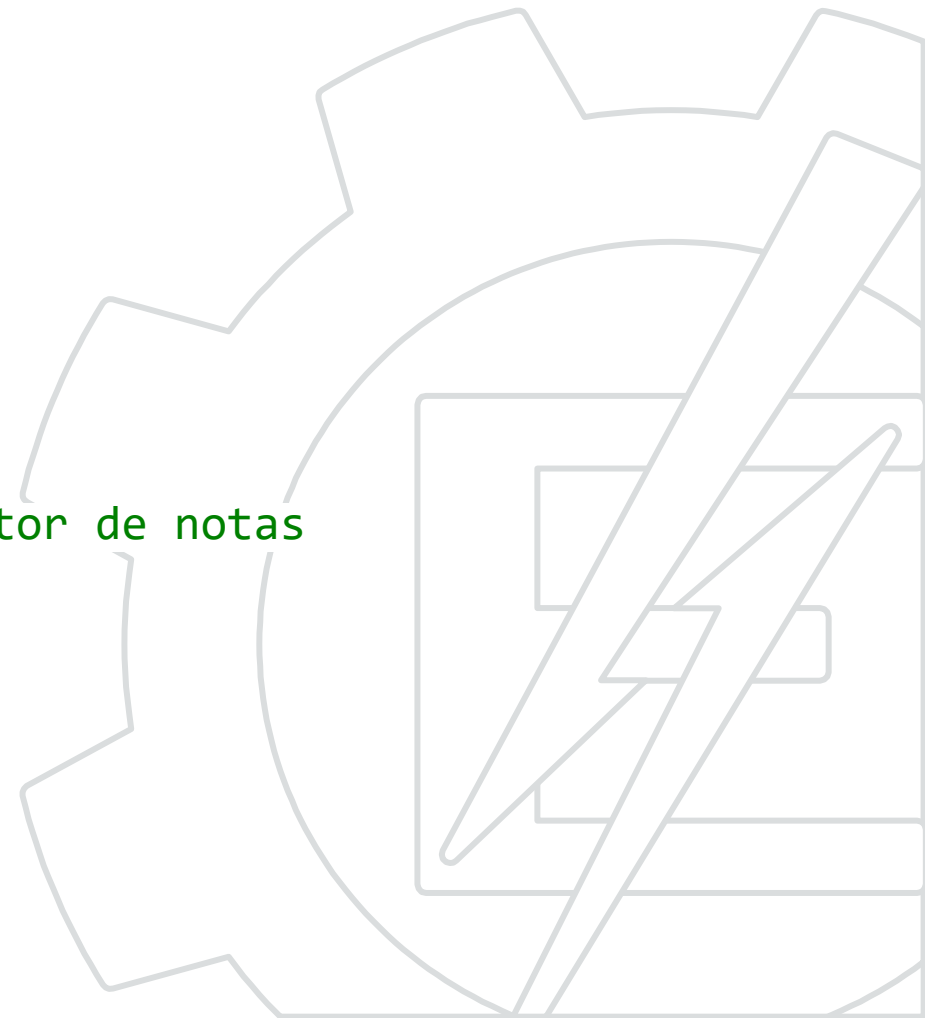


```
void main(void){
    //inicialização
    unsigned char cont=0;
    unsigned char pos=0;
    //Ground Theme (Koji Kondo)
    unsigned char tempo[] = {15, 5, 15, 7, 30, 15, 30, 30, 30, 30, 30,
        30,15, 30, 15, 30, 15, 30, 30, 15, 30, 22, 15, 15, 30, 15, 30, 30,
        15, 15,30, 15, 30, 15, 30, 15, 30, 15, 30, 30, 15, 30, 22, 15, 15,
        30, 15, 30,30, 15, 15, 30, 30, 15, 15, 15, 15, 15, 15, 15, 15,
        15, 15, 15, 15, 15, 30, 15, 15, 15, 15, 15, 15, 15, 15, 15, 15,
        30, 15, 15, 15, 15,15, 15, 15, 15, 15, 15, 15, 15, 15, 15, 30, 15,
        30, 15, 30, 15};
    unsigned int notas[] = {E2, v, E2, v, E2, C2, E2, G2, v, G, v, C2,
        v, G,v, E, v, A, B, AS, A, G, E2, G2, A2, F2, G2, E2, C2, D2, B, v,
        C2, v, G,v, E, v, A, B, AS, A, G, E2, G2, A2, F2, G2, E2, C2, D2,
        B, v, G2, F2S,F2, D2S, v, E2, v, G2, A, C2, v, A, C2, D2, v, G2,
        F2S, F2, D2S, v, E2, v, C3, v, C3, C3, v, G2, F2S, F2, D2S, v, E2,
        v, GS, A, C2, v, A, C2,D2, v, D2S, v, D2, v, C2} ;
    pwmInit();
    timerInit();
    //cont.
```

```
void main(void){
    //inicialização
    unsigned char cont=0;
    unsigned char pos=0;
    //Imperial March (John Williams)
    unsigned char tempo[] = {50, 10, 50, 10, 50, 10, 50, 5, 25, 5, 50,
5, 50, 5, 25, 5, 50, 50, 50, 10, 50, 10, 50, 10, 50, 5, 25, 5, 50,
50, 5, 25, 5, 50, 50, 100, 5, 25, 5, 25, 10, 100, 5, 50, 5, 25,
10, 2, 100, 250};
    unsigned int notas[] = {G, v, G, v, G, v, E, v, B, v, G, v, E, v,
B, v, G, v, D2S, v, D2S, v, D2S, v, E2, v, B, v, FS, v, E, v, B, v,
v, G2S, v, G, v, G, v, G2S, v, G2, v, F2S, v, F2, v, E2, v, F2S, v;
    pwmInit();
    timerInit();
    //cont.
```



```
//inicializa com a primeira nota
pos = 0, cont = 0;
pwmBuzzer(notas[0]);
for(;;){
    //cada ciclo consome 10ms
    timerRest(10000);
    cont ++;
    if (cont >= tempo[pos]){
        //passado o tempo muda a nota
        pos++;
        pwmBuzzer(notas[pos]);
        cont=0;
    }
    //verifica se terminou de percorrer o vetor de notas
    if(pos>=100){
        pos = 0;
    }
    timerWait();
}
```



Até a próxima!

